

[TCW] — Third-Party Research — TCW-73

Transactional email delivery for RelayForge

Vendor research as of Aug 5, 2026 — Decision record for engineering review

Issue: [TCW-73 — Research a transactional email delivery integration](#)

Project Overview

RelayForge is a multi-tenant B2B operations platform for returns, warranty claims and shipment exceptions. The decision is a provider-neutral transactional delivery adapter for customer case notifications—not a marketing or bulk-email system. The synchronous API records durable intent in the business transaction; a NestJS/BullMQ communications worker owns provider handoff and later event reconciliation.

The launch serves roughly 180 merchants in Canada and the United States, with a future EU path. The ticket plans account-verification, return-authorization, shipping-label-ready, refund-status, warranty-case and human-agent notices. Provider-specific types and failures stay behind TransactionalMessagePort; PostgreSQL remains the provider-neutral record.

Source boundaries: the issue description and acceptance criteria define the ticket requirements below. The linked [RelayForge Transactional Email Integration — Research Source Pack](#) is analyzed separately as design constraints. Repository facts come only from the pinned snapshot identified in Evaluation Methodology. The Jira project description was not exposed by the connected project metadata: Not specified in source.

Feature Requirements

Ticket description and acceptance criteria

Functional

- Send the six named customer transactional notice families; exclude marketing and bulk sends.
- Support versioned HTML and plain-text templates in en and fr-CA, tenant branding, metadata, and attachments or time-limited links.
- Record accepted, delivered, deferred, bounced, complained and rejected states in the case audit; support tenant-specific sender identities and a per-tenant disable switch.
- Provide isolated test/sandbox behavior and normalize provider failures and events without leaking vendor fields into the application contract.

Non-functional

- Handle 35,000 messages per business day, normal 25 submissions/s and a 90/s peak for ten minutes.
- Use strict TypeScript on Node 22 in NestJS 11 and BullMQ; deploy on Linux/AMD64 ECS Fargate behind private NAT with no persistent disk.
- Use abortable five-second outbound deadlines, bounded asynchronous retries, idempotency, and duplicate/out-of-order callback tolerance.
- Keep synchronous API and frontend off the provider path; PostgreSQL outbox is the system record and the worker implements TransactionalMessagePort.

Security

- Separate environment credentials and failure classifications; validate authenticated callbacks and make them tenant-scoped and idempotent.
- Assess credential rotation, webhook authenticity, privacy, auditability and safe handling of attachments or links.
- Do not expose provider secrets or recipient/content data through browser code, logs, metrics, traces or synchronous API calls.

Compliance

- Assess data location, retention, privacy terms, suppression/erasure and future EU readiness; the ticket does not prescribe a named certification or exact statutory retention period.

Linked source constraints (not ticket-derived)

Source: RelayForge Transactional Email Integration — Research Source Pack, in the confirmed research folder. No direct issue attachment or remote issue link was returned. The source pack supplies implementation detail; it does not silently expand the ticket-only grouping above.

Implementation restriction — write provider-neutral outbox intent in the business transaction, enqueue BullMQ attempts, and accept callbacks only after raw-body authenticity/replay checks and durable enqueue. No provider fields in case or tenant tables.

Functional/UI dependency — localized audited templates, searchable case history without raw content for unauthorized users, 180 isolated sender identities and a per-tenant kill switch. UI and synchronous API only observe normalized state.

Security/compliance — TLS 1.2+, encryption at rest, least privilege, quarterly rotation, dual-key overlap, environment-separated endpoints, signed/replay-protected callbacks, CA/US processing terms and an EU path. Target 30-day provider operational metadata and 13-month provider-neutral audit; validate erasure and tenant isolation.

Performance — synchronous queue p95 below 150 ms; handoff p95 below 60 s and p99 below five minutes during peak; bounded exponential jitter and per-tenant/environment concurrency; transient retries up to six hours; status visible within two minutes.

Integration assumption — 512 KiB raw webhook limit; reject events older than five minutes absent stronger replay controls; dedupe source plus provider event ID; unknown events cannot mutate case state; suppress permanent failures and complaints. Local Docker Compose uses sandbox only.

Inaccessible linked material: none returned by the issue attachment and remote-link queries. No out-of-folder or external linked document was opened.

Technology Stack

Configuration takes precedence over narrative documentation. Every item below is from the pinned repository snapshot; absence is stated explicitly.

Area	Detected value / decision implication
Frontend	Next.js ^15.3.3, React ^19.1.0, TypeScript; TanStack Query and openid-client. No direct email-provider call.
Backend / worker	NestJS ^11.1.3 REST/OpenAPI; separate NestJS/BullMQ ^5.53.2 worker; Prisma ^6.9; strict TypeScript 5.7.3.
Data / storage	PostgreSQL 16 RDS Multi-AZ; Redis 7 / ElastiCache; S3 with KMS and signed URLs; append-only case events and transactional outbox.
Runtime / package/build	Node >=22.14.0 <23 (container/CI 22.15.1); pnpm 10.12.1 via Corepack; Turborepo 2.5.4; Docker Compose v2.
Authentication	OIDC/OAuth 2.1 Authorization Code + PKCE, signed tokens, server-side session and tenant RBAC. Provider authentication remains adapter-local.
Deployment / cloud	AWS ECS Fargate Linux/AMD64, private subnets/NAT, ALB, CloudFront, WAF, RDS, ElastiCache, S3, Secrets Manager, CloudWatch and OpenTelemetry; Terraform-managed deployment is described, but Terraform source is not present.
CI/CD	GitHub Actions: format, lint, typecheck, tests, build, dependency/container scanning and Trivy; signed immutable ECR image, staging contract/smoke tests, approved digest promotion and ECS migration.
Existing integrations	Stripe, Avalara and Shippo behind adapter boundaries; no email provider SDK is declared.
Coding style	Strict TypeScript including noUncheckedIndexedAccess, ESLint, Prettier, conventional commits, structured JSON, OpenTelemetry, explicit deadlines/retries/idempotency, ports/adapters and ADRs.
Unspecified	Other package ecosystems, Kubernetes/Helm, Azure DevOps and GitLab CI: Not specified in source.

Evaluation Methodology

Repository snapshot: default branch main at commit

8b4c93b20b2949ea95414e81b95fe0f1c10f0fe1. All repository content reads used that SHA.

Vendor research is as of Aug 5, 2026 ; no later-dated source is used.

Exactly eight transactional-email candidates were screened with three independent adoption signals: the official SDK/client repository star display, a fixed seven-day npm registry count, and a third-party category inclusion. The npm interval is Jul 29, 2026 through Aug 4, 2026 . Linked labels resolve to the exact observed URL; observations were captured on the as-of date. A web archive copy was not available in this run, so the dated capture is the snapshot record. Stars shown with “k” preserve GitHub’s displayed rounding; the AWS repository is cross-service and is not an SES-only popularity measure.

Candidate	Official SDK stars	npm seven-day downloads	Third-party inclusion
Amazon SES	3.7k — repository	2,633,360 — registry	G2 transactional email
Twilio SendGrid	3.1k — repository	3,629,292 — registry	G2 transactional email
Mailgun	550 — repository	823,247 — registry	G2 transactional email
Postmark	358 — repository	928,009 — registry	G2 transactional email
Mailchimp Transactional	54 — repository	158,952 — registry	G2 transactional email (free)
SparkPost	177 — repository	34,225 — registry	G2 transactional email
Brevo	120 — repository	188,877 — registry	G2 transactional email
Resend	939 — repository	7,833,218 — registry	StackShare email alternatives

Preliminary selection uses feature set 40%, technology compatibility 30%, documentation 15% and price 15%, each on a ten-point scale. Adoption signals establish a verifiable candidate pool; they do not replace requirement fit. Preliminary estimates for the three candidates not selected are screening judgments with limited evidence, not full vendor reviews.

Candidate	Feature	Compatibility	Docs	Price	Weighted
-----------	---------	---------------	------	-------	----------

Mailgun	9	8	8	9	8.55
Twilio SendGrid	9	8	9	8	8.55
Amazon SES	8	8	9	10	8.45
Resend	8	8	8	8	8.00
Postmark	7	8	9	9	7.90
Brevo	7	7	7	6	6.85
Mailchimp Transactional	7	6	6	5	6.25
SparkPost	6	5	5	5	5.40

Mailgun and SendGrid tie at the top of the screen; both advance. There is no tie at fifth position. The lower-adoption but strongly documented Postmark advances on transactional focus; AWS's broad-repository stars are discounted qualitatively. The detailed review is exactly Amazon SES, Mailgun, Twilio SendGrid, Resend and Postmark.

Final cost normalization is linear within the five observed baseline estimates: $\text{cost score} = 10 - 9 \times (\text{cost} - \$1.00) / (\$20.00 - \$1.00)$. Overall = 0.25 feature + 0.20 compatibility + 0.10 complexity (Easy 10, Medium 6, Complex 3) + 0.15 normalized cost + 0.10 SDK + 0.10 documentation + 0.10 security. Scalability and community are decision-visible columns but not additional overall weights. Ties use lower monthly cost.

Comparison Matrix

Baseline: 10,000 MAU is modeled as one transactional email per active user per month (10,000 sends), no attachment payload, optional add-ons or tax. Vendors bill sends, not MAU. The ticket’s 35,000/business-day production profile is separately modeled in Cost Analysis.

Platform	Feature Coverage (1-10)	Tech Stack Compatibility (1-10)	Integration Complexity	Estimated Monthly Cost (USD)	SDK Quality (1-10)	Documentation Quality (1-10)	Security (1-10)	Scalability (1-10)	Community Support (1-10)	Overall Score
Amazon SES	8	8	Complex	\$1.00	10	9	10	6	10	8.30
Mailgun	9	8	Medium	\$15.00	9	8	10	9	7	7.66
Twilio SendGrid	9	8	Medium	\$19.95	8	9	10	10	9	7.30
Resend	7	8	Easy	\$20.00	9	8	9	3	9	7.10
Postmark	6	8	Medium	\$15.00	8	9	7	5	7	6.61

The five rows above are the complete detailed comparison. Cost normalization scores are SES 10.00, Mailgun/Postmark 3.37, SendGrid 1.02 and Resend 1.00. Estimates are plan-entry points, not quotes for 180 tenants or the production peak.

Recommended Platform

Primary recommendation: Amazon Simple Email Service (SES) v2. It has the strongest weighted result because it fits the existing AWS/Fargate/IAM operating model, has a current modular TypeScript SDK, documented event publishing and very low send-unit cost. It deliberately earns “Complex”: SES requires region-specific identity, quota, event-destination and SNS verification design rather than a single turnkey signed webhook.

Expected implementation: 12–16 engineer-days, including a quota/identity/privacy spike, adapter and contract tests, raw callback ingress, normalization, observability, load test and staged rollout. Expected long-term maintenance is 1–2 engineer-days per month plus metered sends, payload, monitoring and any selected SES plan/add-ons. This is an engineering estimate, not a vendor quotation.

- Gate production on written quota approval and a measured 90/s peak in the chosen region; default sandbox and account quotas are not sufficient evidence.
- Treat ambiguous send errors as an exactly-once risk. Persist intent/tag and reconcile before a resend; SES documents that a rare error can occur after acceptance.
- Validate CA/US processing, future EU path, retention, sender identity and configuration drift with security/legal before tenant onboarding.

Alternatives: Mailgun is the managed choice when direct HMAC-signed webhooks and an EU API region matter more than AWS-native economics. Twilio SendGrid is the enterprise-throughput choice when contractual Mail Send API availability, subuser tooling and support justify a higher plan.

Individual Platform Research

Amazon Simple Email Service (SES) v2

Amazon SES is AWS's regional, metered transactional email service. Its modular JavaScript v3 client, IAM/SigV4 authentication and event destinations align with RelayForge's AWS worker, while identity, quotas and SNS callback composition create a more involved integration than a turnkey webhook provider.

Requirement coverage

- Supports simple, raw and templated sends, HTML/text content, tags/metadata, raw attachments and verified senders; tenant branding, locale/version governance and time-limited S3 links remain application responsibilities.
- Event destinations expose send, reject, bounce, complaint, delivery and delivery-delay signals. Normalize these to the seven ticket states and contract-test deferred/rejected semantics; provider acceptance is not proof of final delivery.
- Regional quotas, sandbox exit and per-second rate are account-specific. A 90/s peak and >35,000/day must be approved and load-tested; sandbox 200/day and 1/s is not a production fit.
- Five-second abort, bounded queue retry, tenant disable, duplicate/out-of-order processing, privacy retention and audit are adapter/application controls. SES documents a rare error after acceptance, so blind retry can duplicate a logical message.

Technology compatibility

- Yes — Node 22.15/strict TypeScript 5.7, NestJS 11 dependency injection and BullMQ 5 worker: the SESv2 adapter can be isolated behind TransactionalMessagePort with explicit timeout/retry policy.
- Yes — Next.js 15/React 19, TanStack Query and the synchronous Nest REST/OpenAPI API remain provider-independent; neither browser nor synchronous request calls the vendor.
- Yes — PostgreSQL 16/Prisma outbox and audit, Redis 7/ElastiCache queues, and S3/KMS time-limited links remain application-owned; no vendor schema is required in case or tenant tables.
- Yes — Linux/AMD64 ECS Fargate private-NAT HTTP egress, ALB/WAF/CloudFront ingress, Secrets Manager, CloudWatch/OpenTelemetry, pnpm 10/Turborepo, Docker, GitHub Actions and the strict lint/test/container pipeline can host and verify the adapter. Task-role credentials avoid a long-lived static API key; configure regional egress and event ingress explicitly.
- Partial — RelayForge OIDC/OAuth 2.1 PKCE, signed tokens and tenant RBAC authenticate users, not the vendor API. Separate adapter credentials/identity, environment scoping and callback verification are required. This is the one partial stack item; score 8.

SDK, documentation and maturity

Official modular `@aws-sdk/client-sesv2` is TypeScript-compatible and part of AWS SDK for JavaScript v3. The repository documents stable public client interfaces, abort support, migration guidance and examples, supported Node versions and a weekday release pattern; latest stable SDK release v3.1103.0 was published `Aug 4, 2026` . Broad official AWS language coverage exists, but a complete SES-language inventory is Not publicly documented within the selected source cap. Documentation rating 9/10: API reference, runnable examples and migration path are present.

Years in operation: Not publicly documented within the source cap. Enterprise support is available through AWS plans; SES-specific contractual SLA and deprecation notice duration: Not publicly documented within the source cap. API v2 is explicit; SDK public-interface stability and exceptional advance deprecation are documented in the repository. Maintenance is active with the current weekday cadence.

Pricing, authentication and compliance

à-la-carte outbound mail is \$0.10 per 1,000 messages plus \$0.12/GB outgoing data. The 10,000-send baseline is \$1.00 before payload and optional tenant, dedicated-IP, global endpoint or plan fees. New SES pricing plans launched `Jul 21, 2026` ; new accounts default to Essentials and may switch to à-la-carte. The old SES-specific 3,000-charge free tier is not available to new customers after that change; eligible new AWS accounts may have general credits. Confirm plan eligibility and regional quote before procurement.

Authentication is IAM/SigV4 with temporary ECS task-role credentials; recipient transport may be opportunistic unless enforced. AWS documents TLS 1.2 for the API, encryption at rest and GDPR resources; SES is listed in AWS SOC scope. SNS HTTP callbacks require certificate/SignatureVersion validation and topic/type allowlisting rather than a provider HMAC header.

Public authentication sample (official SDK pattern; no secret):

```
import { SESv2Client } from '@aws-sdk/client-sesv2';
const ses = new SESv2Client({ region: process.env.AWS_REGION }); // ECS task role /
default provider chain
```

Integration complexity: Complex — regional identity and quota setup, configuration-set event publishing, SNS signature verification and ambiguous-acceptance reconciliation add infrastructure and contract work.

Advantages

- AWS-native IAM, Fargate and observability alignment.
- Very low metered baseline and modular actively maintained SDK.
- Documented event publishing and regional resource boundaries.

Disadvantages

- More moving parts than direct signed webhooks.
- Quotas and sandbox exit are account/region gates.
- Provider acceptance ambiguity complicates exactly-once application semantics.

Risks

- Quota/ramp may miss 90/s; obtain approval and prove it in staging.
- An accepted request followed by an error may duplicate on retry; reconcile intent/tag before resend.
- Regional identity, retention and privacy configuration may drift; audit per tenant/environment.

Implementation steps

1. Create separate AWS accounts/SES regional environments and request sandbox exit/quota headroom.
2. Verify tenant sender domains/identities and DKIM; define configuration sets and event destinations.
3. Grant least-privilege ECS task-role SES/SNS permissions; store non-IAM verification configuration in Secrets Manager.
4. Install and lock @aws-sdk/client-sesv2; security-review dependency changes.
5. Configure region, five-second abort, bounded SDK/queue retries, per-tenant concurrency and kill switch.
6. Implement TransactionalMessagePort in the communications worker; keep frontend/API provider-neutral.
7. Expose raw-body callback ingress; validate SNS certificate, signature, topic/type and replay before durable enqueue.
8. Contract-test locale/template, attachment/link, failure mapping, duplicates, out-of-order and mailbox simulator behavior.
9. Instrument OTEL/CloudWatch queue, handoff, event, throttle, signature and DLQ metrics without content/recipient values.
10. Canary deploy immutable ECS image, load-test quota, enable tenant-by-tenant and retain rollback/disable control.

Official source ledger

Eight numbered source units maximum. Linked labels are the exact pages used; SDK release history is part of its official repository source unit. Each was observed on the vendor-research as-of date. An archived copy was not available, so the dated observation in Evaluation Methodology is the snapshot.

11. [AWS SDK for JavaScript v3 repository and release history](#)
12. [SES pricing](#)
13. [SES pricing-plan change](#)
14. [SES v2 SendEmail API](#)

15. [SES event destinations](#)
16. [SES data protection](#)
17. [SES sending quotas](#)
18. [SNS HTTP signature verification](#)

Mailgun

Mailgun is a transactional API with US and EU endpoints, domain-level sending and directly signed event webhooks. It is a strong managed alternative when regional endpoint selection and an HMAC callback contract outweigh the AWS-native cost advantage.

Requirement coverage

- The Node SDK supports API mail, templates, MIME/attachments and domain configuration; localized versioning, branding, audit state and tenant kill switch remain application-owned.
- Webhooks cover accepted, delivered, temporary/permanent failure and complaint classes; map deferred/rejected and duplicate/out-of-order semantics through contract tests.
- US and EU base URLs separate message/event data by region, while account/API-key material is globally replicated according to the API overview. Review this distinction against CA/US and future EU privacy requirements.
- The enterprise performance SLA documents high throughput; quota and chosen plan must still be approved for the ticket's 90/s peak and 180 tenant domains.

Technology compatibility

- Yes — Node 22.15/strict TypeScript 5.7, NestJS 11 dependency injection and BullMQ 5 worker: the mailgun.js adapter can be isolated behind TransactionalMessagePort with explicit timeout/retry policy.
- Yes — Next.js 15/React 19, TanStack Query and the synchronous Nest REST/OpenAPI API remain provider-independent; neither browser nor synchronous request calls the vendor.
- Yes — PostgreSQL 16/Prisma outbox and audit, Redis 7/ElastiCache queues, and S3/KMS time-limited links remain application-owned; no vendor schema is required in case or tenant tables.
- Yes — Linux/AMD64 ECS Fargate private-NAT HTTP egress, ALB/WAF/CloudFront ingress, Secrets Manager, CloudWatch/OpenTelemetry, pnpm 10/Turborepo, Docker, GitHub Actions and the strict lint/test/container pipeline can host and verify the adapter. US/EU endpoint is explicit configuration; NAT egress and raw webhook ingress are ordinary HTTPS.
- Partial — RelayForge OIDC/OAuth 2.1 PKCE, signed tokens and tenant RBAC authenticate users, not the vendor API. Separate adapter credentials/identity, environment scoping and callback verification are required. This is the one partial stack item; score 8.

SDK, documentation and maturity

Official mailgun.js requires Node 18 or later, uses form-data and ships TypeScript; the repository is active, with latest stable v13.3.0 published Jul 9, 2026. Official Node installation and runnable Basic-auth sample are documented. Broader supported-language inventory, example-repository catalog, migration guide and deprecation notice period: Not publicly documented within the source cap. Documentation rating 8/10: SDK quickstart and API reference are strong; the migration evidence is incomplete.

Years in operation: Not publicly documented within the source cap. Enterprise support and a performance SLA are available on qualifying plans. The API uses URI versioning; a formal deprecation policy is Not publicly documented within the source cap. Recent stable release and TypeScript-dominant repository evidence support an active SDK cadence.

Pricing, authentication and compliance

Free is 100 emails/day with one domain. Basic is \$15/month for 10,000 emails and one domain, with \$1.80/1,000 overage; Foundation starts at \$35 for 50,000 and up to 1,000 domains; Scale starts at \$90 for 100,000. The comparison uses the nominal \$15 10,000-send baseline, but it does not fit 180 tenant sender domains; Foundation \$35 or a negotiated higher-volume plan is the realistic entry point. Enterprise pricing is custom.

The API uses HTTP Basic authentication with user “api” and an API key. Webhooks use HMAC-SHA256 over timestamp and token, with token/timestamp replay controls and optional TLS client certificate. Mailgun publicly documents SOC 2 Type II, ISO 27001, GDPR, TLS and AES-256 at rest.

Public authentication sample (official Node pattern; no secret):

```
import Mailgun from 'mailgun.js'; import FormData from 'form-data';
const mg = new Mailgun(FormData).client({ username: 'api', key:
process.env.MAILGUN_API_KEY! });
```

Integration complexity: Medium — a typed HTTPS SDK and direct HMAC webhook fit the worker, while domain/region and tenant-plan design need explicit controls.

Advantages

- Direct signed webhook and documented replay inputs.
- US/EU API endpoint choice and strong compliance posture.
- Recent TypeScript SDK maintenance and enterprise performance option.

Disadvantages

- Nominal Basic plan has only one domain.
- Account/API-key replication is distinct from regional message data.
- Migration/deprecation details were not established within the source cap.

Risks

- Tenant/domain plan mismatch; validate Foundation or enterprise quote.
- Region selection or globally replicated account data may violate an assumed residency boundary; obtain legal review.
- Webhook token replay and unknown-event mapping require durable dedupe/contract tests.

Implementation steps

19. Create isolated Mailgun environments and choose US/EU data endpoint with legal review.
20. Verify tenant domains and sender identities; confirm plan/domain limits.
21. Generate least-privilege API keys and webhook signing configuration per environment in Secrets Manager.
22. Install and lock mailgun.js and form-data; review dependency security.
23. Configure endpoint, abort deadline, bounded retries, tenant concurrency and disable switch.
24. Implement worker adapter and provider-neutral errors; frontend/API never invoke Mailgun.
25. Receive raw webhook, verify HMAC/timestamp/token and enqueue durably before acknowledgment.
26. Test sandbox, locale/template, attachment/link, failure mapping, duplicate/out-of-order and rate behavior.
27. Instrument sanitized queue, send, event, throttle, signature and DLQ telemetry.
28. Canary ECS deployment and tenant rollout after capacity, privacy and recovery acceptance.

Official source ledger

Eight numbered source units maximum. Linked labels are the exact pages used; SDK release history is part of its official repository source unit. Each was observed on the vendor-research as-of date. An archived copy was not available, so the dated observation in Evaluation Methodology is the snapshot.

29. [mailgun.js repository and release history](#)
30. [Node SDK documentation](#)
31. [Mailgun pricing](#)
32. [API overview and regional endpoints](#)
33. [Security and compliance](#)
34. [Webhook security](#)
35. [Webhook event overview](#)
36. [Enterprise performance SLA](#)

Twilio SendGrid

Twilio SendGrid is a mature transactional Mail Send API with dynamic templates, event webhooks, subuser tooling and enterprise support. It provides a strong contractual-throughput alternative, though the tenant-isolation and plan choice must be validated above the entry-tier estimate.

Requirement coverage

- Mail Send and dynamic templates support transactional content and metadata; locale/version governance, tenant branding and provider-neutral audit remain application controls.
- Event webhook and signed raw-payload verification are available. Validate all seven normalized state mappings, duplicate/out-of-order behavior and durable-ack semantics in contract tests; full event mapping was not established within the source cap.
- Rate-limit headers and 429 behavior are documented per endpoint. The qualifying API SLA states a 15,000/s Americas successful-connection rate, comfortably above the peak; plan and tenant limits still require confirmation.
- EU data residency and subuser management appear on higher plans; procurement must establish the 180-tenant isolation, retention and regional scope rather than assume Essentials provides them.

Technology compatibility

- Yes — Node 22.15/strict TypeScript 5.7, NestJS 11 dependency injection and BullMQ 5 worker: the @sendgrid/mail adapter can be isolated behind TransactionalMessagePort with explicit timeout/retry policy.
- Yes — Next.js 15/React 19, TanStack Query and the synchronous Nest REST/OpenAPI API remain provider-independent; neither browser nor synchronous request calls the vendor.
- Yes — PostgreSQL 16/Prisma outbox and audit, Redis 7/ElastiCache queues, and S3/KMS time-limited links remain application-owned; no vendor schema is required in case or tenant tables.
- Yes — Linux/AMD64 ECS Fargate private-NAT HTTP egress, ALB/WAF/CloudFront ingress, Secrets Manager, CloudWatch/OpenTelemetry, pnpm 10/Turborepo, Docker, GitHub Actions and the strict lint/test/container pipeline can host and verify the adapter. API-key HTTPS egress and ECDSA/OAuth webhook ingress fit the private worker/ALB path.
- Partial — RelayForge OIDC/OAuth 2.1 PKCE, signed tokens and tenant RBAC authenticate users, not the vendor API. Separate adapter credentials/identity, environment scoping and callback verification are required. This is the one partial stack item; score 8.

SDK, documentation and maturity

Official @sendgrid/mail belongs to sendgrid-nodejs; the latest stable repository release 8.1.6 was published Sep 19, 2025 . Twilio documents seven helper languages (C#, Go, Java, Node, PHP, Python and Ruby), a runnable Node quickstart, API reference and repository migration

guide. Documentation rating 9/10. The slower recent release date relative to AWS/Mailgun/Resend and maintained but older cadence yields SDK quality 8/10.

Years in operation: Not publicly documented within the source cap. Enterprise/Premier support and a published Mail Send API SLA are available subject to plan/conditions: 99.95%, or 99.99% with a qualifying package. API v3 is explicit; a formal deprecation notice policy is Not publicly documented within the source cap.

Pricing, authentication and compliance

Free trial is 100 emails/day for 60 days. Essentials starts at \$19.95/month, Pro at \$89.95 and Premier is custom. The 10,000-send comparison uses \$19.95, but subusers, dedicated IP, EU residency and enterprise support can move the 180-tenant solution to Pro/Premier. Exact high-volume price and quota are Not publicly documented within the source cap.

API requests use a restricted bearer API key. Event webhooks support ECDSA signature over timestamp plus raw payload and optional OAuth 2 client credentials. Twilio documents SOC 2 Type II coverage, TLS 1.2/encryption at rest, GDPR DPA/SCC and optional enforced outbound TLS.

Public authentication sample (official Node pattern; no secret):

```
import sgMail from '@sendgrid/mail';
sgMail.setApiKey(process.env.SENDGRID_API_KEY!);
```

Integration complexity: Medium — a mature SDK and signed webhook reduce adapter work, while plan/subuser isolation, regional controls and event normalization remain design gates.

Advantages

- Published qualifying API availability and high connection-rate SLA.
- Broad language helpers, runnable quickstart and migration guidance.
- Signed webhook and higher-tier subuser/residency controls.

Disadvantages

- Entry estimate may not buy required tenant isolation or residency.
- Recent Node release cadence is slower than the leading active SDKs.
- Exact high-volume price is a quote-dependent plan decision.

Risks

- Plan/subuser assumptions may fail for 180 tenants; obtain written entitlement/quote.
- Raw-payload ECDSA/OAuth verification and replay handling must be contract-tested.
- Metadata/category retention can expose sensitive values; forbid PII in vendor tags and review retention.

Implementation steps

37. Create separated SendGrid accounts/environments and select the tenant/residency plan.
38. Authenticate sender domains and establish subuser/tenant isolation.
39. Generate restricted Mail Send keys and webhook verification/OAuth secrets in Secrets Manager.
40. Install and lock @sendgrid/mail; review dependencies.
41. Configure five-second timeout, failure classes, bounded retry and tenant concurrency.
42. Implement worker port adapter and keep frontend/API provider-neutral.
43. Configure event webhook; preserve raw body and validate ECDSA timestamp/signature or OAuth before durable enqueue.
44. Test trial/sandbox, locale/templates, attachments/links, state mapping, duplicates, out-of-order and 429.
45. Add sanitized OTEL/CloudWatch send, queue, event, verification and DLQ monitoring.
46. Canary ECS rollout only after SLA/plan, privacy, quota and recovery approval.

Official source ledger

Eight numbered source units maximum. Linked labels are the exact pages used; SDK release history is part of its official repository source unit. Each was observed on the vendor-research as-of date. An archived copy was not available, so the dated observation in Evaluation Methodology is the snapshot.

47. [sendgrid-nodejs repository and release history](#)
48. [Node Mail Send quickstart](#)
49. [Email API pricing](#)
50. [Event webhook security](#)
51. [v3 rate limits and helper languages](#)
52. [Twilio security overview](#)
53. [GDPR resources](#)
54. [Twilio APIs SLA](#)

Resend

Resend is a developer-focused transactional API with an actively maintained TypeScript/Node SDK, an explicit idempotency-key header and Svix-style signed webhooks. It is simple to integrate, but its default rate limit and US metadata storage are material gates for this ticket.

Requirement coverage

- The send API supports templates, attachments and a 24-hour Idempotency-Key; localized versioning, branding, tenant isolation, durable audit and suppression remain application responsibilities.
- Raw-body signature verification is documented. Duplicate/out-of-order event normalization and durable acknowledgment must be verified in contract tests; do not infer exactly-once delivery from the send idempotency window.
- Default API limit is five requests/s/team, below the 90/s peak. A written increase and measured load test are mandatory before selection for production.
- Regional sends are available, but account metadata, logs and API records are stored in the US regardless of send region. This is a future-EU/strict-residency gap.

Technology compatibility

- Yes — Node 22.15/strict TypeScript 5.7, NestJS 11 dependency injection and BullMQ 5 worker: the resend adapter can be isolated behind TransactionalMessagePort with explicit timeout/retry policy.
- Yes — Next.js 15/React 19, TanStack Query and the synchronous Nest REST/OpenAPI API remain provider-independent; neither browser nor synchronous request calls the vendor.
- Yes — PostgreSQL 16/Prisma outbox and audit, Redis 7/ElastiCache queues, and S3/KMS time-limited links remain application-owned; no vendor schema is required in case or tenant tables.
- Yes — Linux/AMD64 ECS Fargate private-NAT HTTP egress, ALB/WAF/CloudFront ingress, Secrets Manager, CloudWatch/OpenTelemetry, npm 10/Turborepo, Docker, GitHub Actions and the strict lint/test/container pipeline can host and verify the adapter. HTTPS API key, raw signed webhook and container-safe SDK fit; explicit quota increase is external to stack compatibility.
- Partial — RelayForge OIDC/OAuth 2.1 PKCE, signed tokens and tenant RBAC authenticate users, not the vendor API. Separate adapter credentials/identity, environment scoping and callback verification are required. This is the one partial stack item; score 8.

SDK, documentation and maturity

Official resend-node is actively maintained, with latest stable v6.18.1 published Jul 28, 2026 and examples for Node/Express/Next. The send and webhook references provide runnable samples. A complete supported-language inventory, migration guide, API versioning/deprecation policy and years in operation are Not publicly documented within the

source cap. Documentation rating 8/10; SDK quality 9/10 reflects recent TypeScript activity, with the language-policy gap disclosed.

Enterprise support, priority support, flexible retention and an SLA are offered on the custom enterprise tier; the SLA percentage and deprecation notice duration are Not publicly documented within the source cap.

Pricing, authentication and compliance

Free is 3,000/month capped at 100/day. Pro is \$20 for 50,000/month (or \$35 for 100,000); Scale starts at \$90 for 100,000 and overage is \$0.90/1,000; enterprise at three million or more is custom. The baseline comparison is \$20/month. The ticket's peak requires an approved rate increase, not merely a larger monthly allowance.

The API uses an API key; webhook verification uses the raw payload and Svix ID, timestamp and signature. Resend publishes SOC 2 Type II and a GDPR DPA/SCC. Send region selection does not relocate US account metadata/log/API records.

Public authentication sample (official pattern; no secret):

```
import { Resend } from 'resend';
const resend = new Resend(process.env.RESEND_API_KEY!);
```

Integration complexity: Easy — a small typed client, explicit idempotency key and verification helper yield a short adapter path; quota and residency remain production gates.

Advantages

- Explicit 24-hour send idempotency key.
- Recent TypeScript SDK and simple runnable API.
- Signed raw webhook verification helper.

Disadvantages

- Default five/s limit is below the peak.
- US metadata/log storage despite send-region choice.
- Enterprise SLA terms and API deprecation policy are not public within cap.

Risks

- Quota increase may not be granted or sustain 90/s; obtain written approval and test.
- Residency/retention may block EU path; legal review before tenant commitment.
- Idempotency expires after 24 hours; retain provider-neutral intent/dedupe beyond vendor window.

Implementation steps

55. Create isolated environments and confirm quota increase/residency with vendor.
56. Verify sender domains and tenant boundaries.

57. Generate separate API keys and webhook signing secrets in Secrets Manager.
58. Install and lock resend; review dependency security.
59. Configure five-second deadline, Idempotency-Key, bounded retry/concurrency and tenant disable.
60. Implement worker port adapter; frontend/API remain provider-neutral.
61. Configure webhook, preserve raw body and verify Svix signature/timestamp before durable enqueue.
62. Test templates, attachments/links, duplicate/out-of-order events, key expiry, 429 and approved peak.
63. Instrument sanitized queue, handoff, event, verification, throttle and DLQ telemetry.
64. Canary ECS rollout only after quota, privacy, retention and rollback gates pass.

Official source ledger

Eight numbered source units maximum. Linked labels are the exact pages used; SDK release history is part of its official repository source unit. Each was observed on the vendor-research as-of date. An archived copy was not available, so the dated observation in Evaluation Methodology is the snapshot.

65. [resend-node repository and release history](#)
66. [Pricing](#)
67. [Send API and idempotency](#)
68. [Webhook verification](#)
69. [Rate limits](#)
70. [SOC 2](#)
71. [Send regions and metadata location](#)
72. [Data processing agreement](#)

Postmark

Postmark is a focused transactional service with server-scoped credentials, a TypeScript client, sandbox/test token and unusually clear send documentation. Its lack of HMAC webhook signatures and US-only service location make it a weaker fit for the stated callback and future-EU controls.

Requirement coverage

- The API supports HTML/text, templates, metadata and attachments up to 10 MB; locale/version governance, tenant branding, audit and suppression remain application controls.
- The webhook overview explicitly says HMAC signatures are not supported. Basic authentication and IP allowlisting are recommended instead; this does not satisfy the source pack's preferred signed/replay-protected callback without compensating controls.
- The free/test token and sandbox support safe development. Public rate limit and capacity relative to 90/s are Not publicly documented within the source cap.
- Postmark states no own SOC audit and no EU servers planned; data centers are SOC 2 and a GDPR DPA/SCC exists. This is not equivalent to service-level SOC 2 or EU data residency.

Technology compatibility

- Yes — Node 22.15/strict TypeScript 5.7, NestJS 11 dependency injection and BullMQ 5 worker: the postmark adapter can be isolated behind TransactionalMessagePort with explicit timeout/retry policy.
- Yes — Next.js 15/React 19, TanStack Query and the synchronous Nest REST/OpenAPI API remain provider-independent; neither browser nor synchronous request calls the vendor.
- Yes — PostgreSQL 16/Prisma outbox and audit, Redis 7/ElastiCache queues, and S3/KMS time-limited links remain application-owned; no vendor schema is required in case or tenant tables.
- Yes — Linux/AMD64 ECS Fargate private-NAT HTTP egress, ALB/WAF/CloudFront ingress, Secrets Manager, CloudWatch/OpenTelemetry, pnpm 10/Turborepo, Docker, GitHub Actions and the strict lint/test/container pipeline can host and verify the adapter. HTTPS server token and container-safe TypeScript client fit; webhook security requires compensating ingress controls.
- Partial — RelayForge OIDC/OAuth 2.1 PKCE, signed tokens and tenant RBAC authenticate users, not the vendor API. Separate adapter credentials/identity, environment scoping and callback verification are required. This is the one partial stack item; score 8.

SDK, documentation and maturity

Official postmark.js is TypeScript-dominant; latest stable 4.0.7 was published Feb 18, 2026 . The developer portal lists TypeScript/Node, Python, Ruby/Rails, PHP, .NET and Java clients, a

runnable send sample, API reference, sandbox and migration guides. Documentation rating 9/10; SDK quality 8/10 reflects maintained but less frequent releases.

Years in operation, published service SLA, formal API versioning/deprecation notice policy and peak rate commitment: Not publicly documented within the source cap. Support and high-volume contact are offered, but enterprise SLA terms require vendor confirmation.

Pricing, authentication and compliance

Free is 100 emails/month. Basic is \$15/month for 10,000 with \$1.80/1,000 overage, 45-day retention and five custom domains. The comparison uses \$15; five domains do not meet 180 isolated tenant identities, so a higher platform/custom arrangement is required and its price is Not publicly documented within the source cap.

Send authentication uses a server API token; the test token POSTMARK_API_TEST avoids production delivery. Webhook Basic auth and IP allowlisting are documented, but HMAC is explicitly unavailable. GDPR DPA/SCC is published; Postmark distinguishes data-center SOC 2 from its own lack of SOC audit and states the primary service is US-based.

Public authentication sample (official TypeScript pattern; no secret):

```
import { ServerClient } from 'postmark';
const client = new ServerClient(process.env.POSTMARK_SERVER_TOKEN!);
```

Integration complexity: Medium — the send client is straightforward, but callback authenticity/replay compensation, tenant domain plan and residency require additional design.

Advantages

- Clear transactional send API, sandbox and test token.
- Broad official client list and migration guidance.
- Server-scoped credentials and concise TypeScript sample.

Disadvantages

- No HMAC webhook signature.
- No EU service region; service itself has not undergone SOC audit.
- Basic plan permits only five domains and peak rate is not established.

Risks

- Callback spoof/replay risk; require authenticated ingress, IP controls, replay cache and durable dedupe or reject provider.
- Tenant/domain capacity and quote may not fit 180 merchants.
- US location and absence of service SOC audit may fail procurement/EU path.

Implementation steps

73. Create isolated Postmark accounts/servers and obtain tenant/domain and capacity terms.

74. Verify sender domains and streams per approved isolation model.
75. Generate server tokens and ingress Basic credentials per environment in Secrets Manager.
76. Install and lock postmark; review dependencies.
77. Configure deadline, bounded retry, tenant concurrency and disable switch.
78. Implement worker port adapter; frontend/API remain provider-neutral.
79. Apply documented webhook Basic auth/IP controls, replay/dedupe and durable enqueue; document residual unsigned risk.
80. Test POSTMARK_API_TEST/sandbox, templates, attachments, failure/state mapping and duplicate events.
81. Instrument sanitized queue, send, event, ingress and DLQ telemetry.
82. Deploy only after security, privacy, capacity and compensating-control approval; canary tenant rollout.

Official source ledger

Eight numbered source units maximum. Linked labels are the exact pages used; SDK release history is part of its official repository source unit. Each was observed on the vendor-research as-of date. An archived copy was not available, so the dated observation in Evaluation Methodology is the snapshot.

83. [postmark.js repository and release history](#)
84. [Postmark pricing](#)
85. [Send email API guide](#)
86. [Webhook overview and security limitation](#)
87. [Developer portal and clients](#)
88. [EU privacy and SOC clarification](#)
89. [Data processing agreement](#)
90. [GDPR FAQ](#)

Architecture Blueprint

Recommended design: Amazon SES v2. The application-boundary, durability, tenant and SLO choices below are RelayForge design decisions derived from the ticket/source pack.

SES/SNS-specific behavior is limited to the linked public APIs and signature guidance; capacity, privacy and plan assumptions remain explicit gates.

High-Level Integration Architecture

- A case transaction writes a provider-neutral PostgreSQL outbox intent; the synchronous Nest API and Next.js frontend never wait on or call SES.
- A BullMQ communications worker resolves tenant policy and invokes an SESv2 adapter behind TransactionalMessagePort on ECS Fargate.
- SES configuration-set events flow to an SNS HTTPS ingress; raw-body authenticity is checked before durable queueing and a worker normalizes audit state.
- Tenant/environment identity, region, suppression and disable controls remain separate from business tables; no vendor types escape the adapter.

Data Flow

- Persist one logical message ID, tenant, locale/template version, safe metadata and trace context with the business event.
- Before handoff, check tenant disable/suppression, render approved escaped HTML/text and prefer an expiring S3 link over retained payload where feasible.
- Call SES with verified identity, configuration set and non-sensitive correlation tag; persist provider MessageId and attempt outcome without body or raw recipient.
- Deduplicate source plus provider event ID, tolerate out-of-order delivery, normalize accepted/delivered/deferred/bounced/complained/rejected and update the case audit.

Authentication Flow

- Use the ECS task role and AWS SDK default temporary credential provider chain for SigV4 SES requests; scope actions, region and resources to the environment.
- Verify tenant domains/DKIM and separate development, staging and production identities, endpoints and policies; audit changes and rotate any non-IAM secret with overlap.
- For SNS ingress, validate HTTPS SigningCertURL against trusted AWS patterns, canonical signed fields, signature version, TopicArn and message Type before enqueue.
- Enforce a five-minute freshness/replay policy and durable message-ID dedupe; RelayForge OIDC/PKCE and RBAC continue to protect operator/UI access, not SES.

Error Handling Strategy

- Map authentication, identity, configuration, invalid-recipient and documented permanent errors to a sanitized permanent class/DLQ; do not spin on them.

- Map network, throttling and documented transient 5xx errors to bounded asynchronous retry with tenant-aware backpressure.
- Treat an error after possible provider acceptance as ambiguous: retain intent/tag, reconcile before resend and expose an operator-safe state.
- Reject invalid signatures, oversized/old callbacks and unknown events without case mutation; record only redacted reason and correlation.

Retry Mechanism

- Apply an abortable five-second SDK deadline and avoid multiplying opaque SDK retries with BullMQ retries.
- Use bounded exponential backoff with jitter, per-tenant/environment concurrency and a maximum transient retry horizon of six hours.
- Persist attempts and next action in provider-neutral state; send permanent/complaint outcomes to suppression or DLQ rather than retry.
- Provide audited operator replay by logical intent, preserving idempotency and checking ambiguous acceptance before another provider call.

Rate-Limit Handling

- Obtain explicit regional production quota above 35,000/day and 90/s with headroom; sandbox limits are not evidence of readiness.
- Use a Redis-backed token bucket and per-tenant fair concurrency so one merchant cannot consume the environment budget.
- Honor throttling with jittered backoff, reduce concurrency on sustained 429/5xx and alarm on queue age and quota utilization.
- Warm identities and ramp traffic through a measured canary; verify p95 handoff <60 s and p99 <5 min under the ten-minute peak.

Caching Recommendations

- Cache non-secret tenant identity, configuration-set and disable policy briefly with explicit invalidation and an authoritative database fallback.
- Use the AWS credential provider/Secrets Manager refresh model; never cache long-lived secrets in browser code, images or persistent local disk.
- Use a durable unique constraint for callback dedupe and logical intent; Redis may accelerate replay checks but is not the sole correctness store.
- Do not cache bodies, unredacted recipient addresses or sensitive substitutions in shared caches, telemetry or callback records.

Security Considerations

- Require TLS 1.2+ to AWS APIs and decide whether enforced recipient TLS is needed for confidential notices; encrypt application data and S3 objects at rest.

- Limit raw callback bodies to 512 KiB, validate certificate/signature/topic/freshness before durable enqueue, and rate-limit ingress at WAF/ALB.
- Keep payment cards, passwords, tokens and full profiles out of templates; use expiring links and safe escaped substitutions.
- Review CA/US processing, future EU path, 30-day provider metadata target, 13-month neutral audit, erasure/suppression and tenant separation with security/legal.

Monitoring & Logging

- Propagate trace context from outbox to queue attempt and normalized event; emit structured, tenant-scoped correlation without body, secret or raw recipient.
- Measure queue depth/age, handoff latency, accepted/delivered/deferred/bounce/complaint/reject rates, throttles, transient/permanent errors and DLQ age.
- Alarm on signature/replay/unknown-event rejection, quota utilization, p95/p99 SLO breach and missing event reconciliation; expose operator status within two minutes.
- Use CloudWatch/OpenTelemetry dashboards and audited runbooks; restrict tenant drill-down and test redaction in CI.

Deployment Considerations

- Pin and security-review the npm SDK dependency and container; keep Linux/AMD64 Fargate images immutable and non-root.
- Test local Docker Compose against sandbox/mailbox simulator only; exercise port fakes, contract tests, locale/template rendering, webhook signatures and failure mapping.
- Provision identities, quotas, configuration sets, SNS topics and IAM separately per account/region; validate production-like capacity before enabling tenants.
- Use staged ECS canary/digest promotion, expand/contract database migration, feature flag/tenant disable and circuit-breaker rollback.

Public design anchors: [SES SendEmail v2](#) — [event destinations](#) — [sending quotas](#) — [SNS signature verification](#)

Risks & Mitigation

Risk	Mitigation / release gate
Regional quota and sender ramp cannot sustain 90/s.	Written quota approval, staged warm-up, token bucket, load test and SLO alarm before tenant enablement.
Ambiguous provider acceptance causes duplicate customer notices.	Persist one logical intent; tag/reconcile before resend; durable event dedupe and operator replay controls.
Privacy, residency or retention assumptions fail procurement.	Security/legal review of processing terms, region, metadata retention, erasure, suppression and future EU path.
Callback spoof, replay or unknown state mutates a case.	Raw-body certificate/signature/topic/freshness validation, 512 KiB cap, durable dedupe, allowlisted state machine.
Tenant identity/configuration drift or cross-tenant leakage.	Separate environment IAM/identities, audited mapping, contract tests, least privilege, kill switch and periodic review.

Cost Analysis

The comparison baseline is an explicit modeling assumption: 10,000 MAU × one notice/user/month = 10,000 sends. It excludes tax, attachment bytes, dedicated IP, support, inbound mail, monitoring and negotiated discounts. It is not a confidential internal forecast or vendor quote.

Platform	10k-send baseline	180-tenant / production caveat
Amazon SES	\$1.00 à-la-carte outbound	Plus \$0.12/GB and optional plan/tenant/IP/global-endpoint charges; verify new-account plan selection.
Mailgun	\$15 Basic	One domain only; Foundation \$35 is a more plausible domain-capacity floor; high volume/enterprise quote needed.
Twilio SendGrid	\$19.95 Essentials	Subusers, EU residency and support may require Pro \$89.95 or Premier/custom.
Resend	\$20 Pro	Monthly allowance does not remove the default 5/s limit; approved quota and residency review required.
Postmark	\$15 Basic	Five domains only; 180-tenant arrangement and peak capacity require custom terms.

Ticket-volume sensitivity: 35,000 per business day implies approximately 770,000 sends in a 22-business-day month, or 1,050,000 if that rate occurs on all 30 days. SES à-la-carte outbound is approximately \$77 or \$105 respectively before payload and options. Optional SES tenant pricing, if chosen for 180 tenants, adds \$0.90/month tenant resource fee plus approximately \$3.85 or \$5.25 message tenant charges. New plan eligibility, region and discounts must be confirmed.

Illustrative list-price extrapolations are not procurement quotes: Mailgun Scale 100k plus \$1.10/1,000 overage is about \$827/\$1,135; Resend Scale 100k plus \$0.90/1,000 is about \$693/\$945; Postmark Basic plus \$1.80/1,000 is about \$1,383/\$1,887, but its domain tier is ineligible for the assumed tenancy. SendGrid high-volume pricing is Not publicly documented within the source cap. Vendor volume tiers and negotiated rates can materially change every extrapolation.

Implementation Checklist

Release checklist for the primary SES design. These ten items are intended as native checkable list entries in the final Google Doc.

- ☐ Approve regional quota, sandbox exit, 90/s headroom and sender ramp plan.
- ☐ Verify tenant domains/DKIM, configuration sets and environment separation.
- ☐ Approve IAM task roles, Secrets Manager rotation and callback trust policy.
- ☐ Pin/security-review the modular SESv2 SDK and deterministic lockfile.
- ☐ Implement the worker port adapter, five-second abort and provider-neutral failure mapping.
- ☐ Implement raw SNS verification, freshness, durable enqueue and event dedupe.
- ☐ Contract-test locale/template, attachment/link, suppression and ambiguous acceptance.
- ☐ Load-test queue fairness, 90/s peak and p95/p99 handoff SLOs.
- ☐ Ship redacted OTEL/CloudWatch dashboards, alarms, DLQ and operator runbook.
- ☐ Canary by tenant with privacy approval, rollback and audited disable control.

Final Recommendation

Approve Amazon SES v2 as the single primary integration candidate, conditional on quota, identity, callback-security and privacy gates. Its 8.30 weighted score combines AWS-native compatibility, active SDK maintenance, strong documented security and the lowest modeled send-unit cost. The implementation is deliberately a provider-neutral worker adapter so a later substitution does not rewrite case logic.

Plan 12–16 engineer-days for the first production-ready path and 1–2 engineer-days/month for ongoing policy, SDK, delivery, quota and incident maintenance, in addition to metered service and observability costs. Do not enable customer traffic until a measured 90/s quota test, duplicate-safe ambiguous-send handling and CA/US/future-EU review are accepted.

Mailgun is the alternative when direct HMAC webhooks and EU endpoint selection dominate. Twilio SendGrid is the alternative when contractual enterprise throughput, subuser capabilities and support dominate. Neither is an additional primary recommendation.